Data Structures

MIDSEM PRACTICE PAPER | 30 marks | Suggested time: 90 minutes

Modules 1-2; module 3 restricted to stacks

PROVISIONAL mapping: no DS channel is visible in the current team. Module 1 is inferred as C structures, pointers and arrays; module 2 as linked lists. Stacks are included provisionally per your instruction. Trees, graphs and queue-specific questions are excluded.

Answer all questions. Show working and draw labelled diagrams where appropriate. Marks and duration are a practice format, not a confirmed university blueprint.

Question 1 [2 marks]

Write a C program to read and display employee details including the date of joining using nested structure. Consider date of joining as nested structure.

Source: DS-M | original Q1C

Question 2 [2 marks]

Assume that you are storing marks of students in 3 subjects. Write a C program that: i. Takes the number of students as input. ii. Dynamically allocates a 2D array using pointers. iii. Accepts marks for each subject.Calculates and prints the average marks of each student using only pointer arithmetic.

Source: DS-R | original Q2C

Question 3 [2 marks]

Write a C program to dynamically allocate memory for 2D string arrays (array of strings) and perform concatenation using pointers.

Source: DS-M | original Q2C

Question 4 [5 marks]

A music streaming service keeps track of songs in a playlist using a singly linked list. Each node stores the following details: SongID (integer), Title (string), Duration (in seconds). Write a C program to perform the following operations: Insert songs into the playlist in the order they are added. Remove all songs whose duration is less than a user-specified value (Use head pointer for performing this operation). Display the updated playlist. Include a main() function to demonstrate all functionalities.

Source: DS-R | original Q4A

Question 5 [5 marks]

A mobile app named "MIT-BLR Connect" maintains contact details of alumni in three different modules: i) Add Recent Contacts Module: Displays recently interacted alumni in order of contact - new ones added at the beginning. ii) Traverse Module: Allows viewing alumni details in forward direction. iii) Develop a module to print the data in reverse order. Develop C functions considering singly linked list as the suitable data structure

Source: DS-M | original Q4A

Question 6 [5 marks]

A college is developing a simple "Smart Editor" software for its students. In the Smart Editor, students should be able to undo and redo text changes.As a software developer, you are asked to analyze and illustrate how Stacks can be used in this application. Question: i. Explain how a Stack can be used to implement the Undo-Redo feature in the editor.Write short C code snippets for implementation and explain how it will work.

Source: DS-R | original Q2A

Question 7 [5 marks]

Write a function to convert an infix expression to postfix. Include the following functions as per the prototype. i) int InStackPrecedence(char op) ; to return the precedence of operator within the stack ii) int IncomingPrecedence(char op); to return the precedence of the incoming operator. iii) void Infix_Postfix(char infix[], char postfix[]); to convert given infix expression to postfix. Main function not required. Note: The infix expression may include ONLY the following operators (listed in the order of their precedence): • ^ (exponentiation) → right associative • * (multiplication), / (division) → left associative • + (addition), - (subtraction) → left associative

Source: DS-M | original Q2A

Question 8 [2 marks]

Write a C function bool isDuplicate(char s[]) that detects duplicate parentheses in a given expression using a stack.

Source: DS-M | original Q3C

Question 9 [2 marks]

Using the following nested calls, justify why a stack is preferred over a queue in handling function calls. Explain the role of activation records (stack frames).

```
void fun1() { printf("Inside fun1\n"); }
void fun2() { fun1(); printf("Inside fun2\n"); }
int main() { fun2(); printf("Inside main\n"); }
```

Source: DS-R | original Q3C

Sources and selection notes

Working interpretation: NSM means end-semester. This has not yet been confirmed. Questions come only from the local semester-3 CSE papers, using the exact current subject titles. R = regular; M = makeup. Wording is lightly edited, tables transcribed, original question references retained. No new question scenarios were invented.

DS-M: 2025-26 Makeup - CSS_2101-Data Structures

Local source: /Users/shritaake/Desktop/code/paper/public/authoritative/MIT 2021-26/MIT 2025-26 QPs 3rd March 2026/December 2025 and January 2026 (Makeup)/MIT Sem 1 to Sem 7 QPs/Semester 3/CSS_2101-Data Structures.pdf

DS-R: 2025-26 Regular - CSS-2101-Data Structures

Local source: /Users/shritaake/Desktop/code/paper/public/authoritative/MIT 2021-26/MIT 2025-26 QPs 3rd March 2026/November and December 2025 (Regular)/MIT Sem 1 to Sem 7 QPs/Semester 3/CSS-2101-Data Structures.pdf

PROVISIONAL mapping: no DS channel is visible in the current team. Module 1 is inferred as C structures, pointers and arrays; module 2 as linked lists. Stacks are included provisionally per your instruction. Trees, graphs and queue-specific questions are excluded.